

Hyperparameter Searching in NN



View to Co. Pintor · Jan 03, 2026

"Mauricio A. Valle"

1/1100 · f/8.0 · 35mm Fujinon · ISO 200

Preparado por Prof. M.A.Valle
mauricio.valle.b@uai.cl

Qué son parámetros e hiperparámetros

El ajuste de hiperparámetros (**hyperparameter tuning**) es el proceso de seleccionar los valores óptimos del modelo de NN. Esto se hace usualmente antes del proceso de entrenamiento y es iterativo.

Es importante, porque una buena selección de estos hiperparámetros ayuda al modelo a encontrar mejores patrones, se evita overfitting y se logra mejor desempeño.

ACLARACIÓN:

Parámetros no es lo mismo que hiperparámetros.

Los **parámetros** son los pesos sinápticos y biases que se van ajustando automáticamente en el aprendizaje.

Los **hiperparámetros** (learning rate, layers, función de activación, etc) son ajustes EXTERNOS manuales configurados ANTES del entrenamiento.

Por qué es importante

1. Al encontrar una combinación óptima de hiperparámetros, logramos un mejor desempeño del modelo (ej. Mayor accuracy y robustez).
2. Ayuda a evitar el over y under-fitting resultando en un modelo más balanceado.
3. Si hacemos grid search sobre un set de validación, es más probable que logremos un modelo más generalizable.

Desafíos:

1. A veces el espacio de búsqueda es muy grande y no tenemos disponible muchos recursos.
2. A veces no sabemos bien qué hiperparámetros *mover*.

Técnicas de búsqueda de hiperparámetros

Como ya es de su conocimiento, un modelo de NN tiene varios hiperparámetros.

Encontrar la combinación que optimice algún criterio se puede tomar como un clásico problema de búsqueda (optimización) y suele tomar bastante tiempo:

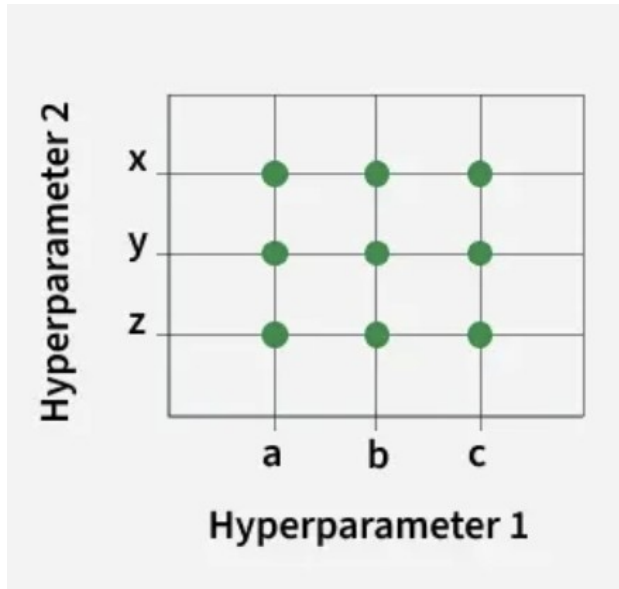
Tiempo del proyecto actividad

60-80%	Data cleaning, EDA.
5-10%	hyperparameter tuning
1-5%	model training

Técnicas de búsqueda de hiperparámetros

1. GRID SEARCH (Veremos este en ejemplo):

Aquí la idea es probar en forma exhaustiva todas las combinaciones de valores predefinidos de los hiperparámetros.



Cada punto verde es una combinación específica de hiperparámetro 1 (a,b,c) e hiperparámetro 2 (x,y,z). Cada posibilidad es testeada.

Técnicas de búsqueda de hiperparámetros

1. GRID SEARCH:

Equivale a utilizar fuerza bruta. Útil cuando la cantidad de hiperparámetros a testear no son tantos, de modo que no se demore tanto, de otro modo, podemos agotar recurso computacional.

Procedimiento:

- a. crear una grilla o la combinación de todas las posibilidades (obviamente antes hay que tener muy claro cuáles son los hiperparámetros a probar y el(los) performance(s) a evaluar).
- b. Entrenar el modelo para cada combinación de la grilla (aquí utilizar datos de validación). Idealmente utilizar cross-validation.
- c. Seleccionar la combinación que de el mayor performance.

Técnicas de búsqueda de hiperparámetros

1. GRID SEARCH:

Por ejemplo, supongamos que tenemos hiperparámetro C y alpha y el accuracy para cada combinación es el siguiente:

Fijarse que son en total $5 \times 4 = 20$ combinaciones a probar.

La mejor resulta en $C=0.3$, y $\alpha=0.2$

Luego con train set entrenamos modelo con esos hiperparámetros.

C	0.5	0.701	0.703	0.697	0.696
	0.4	0.699	0.702	0.698	0.702
	0.3	0.721	0.726	0.713	0.703
	0.2	0.706	0.705	0.704	0.701
	0.1	0.698	0.692	0.688	0.675
		0.1	0.2	0.3	0.4
		Alpha			

Técnicas de búsqueda de hiperparámetros

1. GRID SEARCH:

En Python, podemos hacerlo “a mano” o utilizar GridSearch, por ejemplo en el caso del Airbnb.

Partimos con:

- * `airbnb_datasets_example_student.ipynb` (versión cruda para partir).

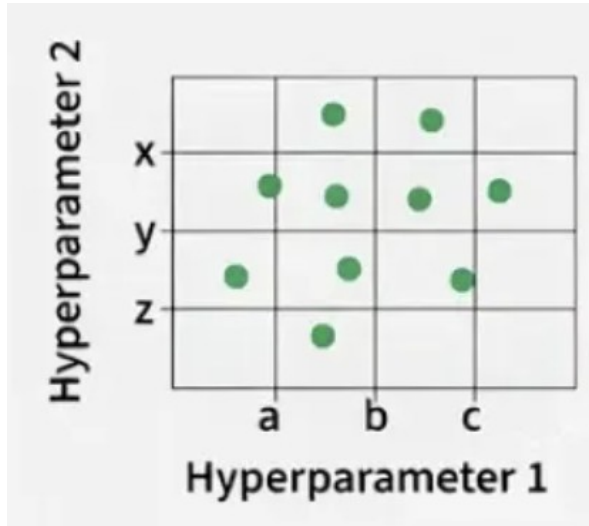
Luego:

- * `airbnb_datasets_example_student4.ipynb` aquí hacemos un Grid Search a mano con Loop.

Técnicas de búsqueda de hiperparámetros

2. RANDOM SEARCH:

Aquí la idea es seleccionar de manera aleatoria combinaciones de hiperparámetros en vez de seleccionarlos todos.



Los puntos verdes se seleccionan al azar en el mismo espacio de parámetros, sin cubrir cada combinación.

Técnicas de búsqueda de hiperparámetros

2. RANDOM SEARCH:

En este caso, podemos programar nosotros mismos una función que dado el set de hiperparámetros a testear los seleccionen en forma aleatoria. Esta alternativa es mejor cuando el espacio de búsqueda es demasiado grande y no tenemos mucho tiempo o recurso computacional.

La otra alternativa es utilizar RandomizedGridSearch de Python que suele ser la mejor para la mayoría de las aplicaciones.

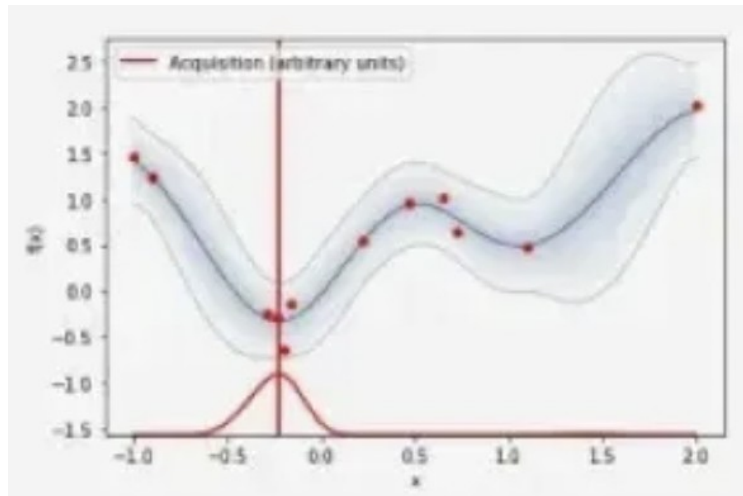
Procedimiento:

- a. definir los hiperparametros y rangos a probar.
- b. se seleccionan en forma aleatoria las combinaciones sin reemplazo y se testea utilizando por ejemplo 5-fold cross validation.
- c. se selecciona la mejor combinación que optimiza el performance previamente seleccionado.

Técnicas de búsqueda de hiperparámetros

2. OPTIMIZACIÓN BAYESIANA:

Aquí la idea es construir un modelo probabilístico para predecir la performance de los hiperparámetros y se elige la mejor combinación.



La curva muestra una predicción de un modelo con incerteza. Los puntos rojos son evaluaciones pasadas (combinaciones testeadas). La línea roja vertical muestra el siguiente punto de test prometedor.

Técnicas de búsqueda de hiperparámetros

2. OPTIMIZACIÓN BAYESIANA:

El random search suele ser ineficiente (en especial cuando el espacio de búsqueda es muy grande) porque trata de “matar la tórtola con escopetazos a ciegas”.

En la optimización Bayesiana, cada hiperparámetro se trata como un problema de optimización matemático aprendiendo de los últimos resultados para decidir qué intentar en el siguiente paso.

Escencialmente, la búsqueda se hace encontrando

$$P(\text{score}(y)|\text{hyperparameter}(x))$$

Este método no lo veremos en detalle aquí en este curso.